Proyecto 3: Pochoco SoC de RTL a GDSII

Macro digital en GF180MCU con LibreLane

Arquitectura de Computadores
Universidad de los Andes
Semestre 2026-2

Integrantes: Nicolás Araneda Vicente Calderón Santiago Muñoz

Noviembre de 2026

Todo el código, las configuraciones y los reportes están en https://github.com/nicolasaraneda/Proyecto3_Arquitectura_De_Computadores. Los nombres de archivo, módulo y corrida son los del repositorio, y las decisiones se citan con su número en `docs/decisiones.md` (D1 a D23). Las corridas usan LibreLane 3.0.16 en Docker, el PDK gf180mcuD (revisión `54435919`) y la biblioteca `gf180mcu_fd_sc_mcu7t5v0`; el README tiene las versiones exactas, la instalación y los comandos.

La corrida final (`final`, `asic/rom/config.yaml`) usa $T = 100$ ns, `FP_CORE_UTIL` $= 40$ %, la estrategia de memoria 1 (Sección 2), el core con lectura del banco de registros y decodificación en el fetch y el modelo de cable medido para el resizer (Sección 4, D23). Se exige cerrar timing en las nueve esquinas que analiza LibreLane: tres de parásitos (min, nom, max) por tres de proceso, voltaje y temperatura (ss a 125 °C y 4,5 V, tt a 25 °C y 5 V, ff a −40 °C y 5,5 V). Entre ellas están la típica, `nom_tt_025C_5v00`, y la más lenta, `max_ss_125C_4v50`. LibreLane solo trata como error la típica, así que el cierre en las demás se verifica con las métricas de cada esquina (`asic/verificar_cierre.py`, D14). La Tabla 1 resume el resultado.

Criterio	Resultado	Peor esquina
DRC (Magic)	0 violaciones	
DRC (KLayout, deck de GlobalFoundries)	0 marcadores en 684 reglas	
LVS (Netgen)	0 diferencias	
Antenas	0 nets (1 diodo insertado)	
Setup: WNS y TNS	65,73 ns y 0 ns	`max_ss_125C_4v50`
Hold: WNS y TNS	0,17 ns y 0 ns	`min_ff_n40C_5v50`

Tabla 1: Criterios de cierre de la corrida final. El hold de `nom_ff_n40C_5v50` empata con el de `min_ff`. Fuente: `cierre.md` y `drc_klayout/resumen.md` en `asic/rom/resultados/final/`.

1 Del RTL de FPGA al RTL de ASIC

`rtl/asic_top.v` instancia `pochoco_soc` sin nada propio de la Go Board. Sus puertos son `clk`, `rst_n` (activo en bajo), `btn[3:0]`, `led[3:0]`, `seg1[6:0]` y `seg2[6:0]` (un display cada uno, en orden {G,F,E,D,C,B,A} y activos en alto) y `spi_sclk`, `spi_mosi`, `spi_cs_n` y `spi_miso`. El reset pasa por un sincronizador de dos flip-flops que lo activa en forma asíncrona y lo libera en un flanco de reloj, así que todos los flip-flops salen del reset en el mismo ciclo (D6). Los parámetros `MemStrategy` y `DataWords` eligen la memoria de la Sección 2, y `EarlyRead` la temporización del core (Sección 4.3).

Para encontrar lo que dependía de la FPGA se corrió LibreLane sobre el RTL del Proyecto 2 (commit `1227fa5`) aplicando los reemplazos de a uno: c0a con el RTL sin cambios, c0b con el banco

de registros en flip-flops y c0c con `asic_top`. La Tabla 2 lista cada construcción con el mensaje del flujo que la evidenció.

Construcción	Mensaje del flujo (archivo:línea)	Reemplazo
BRAM `SB_RAM40_4K` en `espino_register_file.v`	`%Error-MODMISSING ... SB_RAM40_4K`, el linter detiene el flujo (`c0a-verilator-lint.txt:1` y `:13`)	Banco de registros en flip-flops con lectura sincrónica, igual que la BRAM (D3, D4)
Contador de reset de encendido `por_cnt` con valor inicial	`%Warning-PROCASSINIT ... 'por_cnt'` (`c0b/verilator-lint.txt:7`) y `Set init value: \por_cnt` (`c0b/yosys-synthesis.txt:324`)	Puerto `rst_n` con sincronizador en `asic_top` (D6)
`$readmemh` dentro de un `initial` (`pochoco_ram.v:36`)	`Removed top 23 address bits ... from memory init port` (`c0b/yosys-synthesis.txt:992`): Yosys lo convierte en valor inicial de la memoria	ROM y memoria generadas por el assembler (Sección 2)
Inferencia de BRAM para la RAM de 512 palabras	`Adding EN signal on $memory\u_soc.u_ram.mem[93]` (`c0b/yosys-synthesis.txt:1469`): la memoria pasa a flip-flops; `c0b/stat.rpt`: 17 188 flip-flops y 2 327 825 µm^2	Memoria dimensionada según el uso del juego (Sección 2)
SPI sin instanciar	`%Warning-UNUSEDSIGNAL ... 'spi_req'` (`c0b/verilator-lint.txt:18`)	Puertos presentes, `spi_miso` fijo en 1: el juego no usa SPI (D7)

Tabla 2: Construcciones del Proyecto 2 que dependían de la FPGA. Los archivos son copias de los logs de LibreLane en `asic/p2_original/evidencia/`.

Los dos valores iniciales son el problema de fondo. La biblioteca no tiene flip-flops con valor de encendido, así que la síntesis los descarta sin otro aviso que esas líneas. En silicio `por_cnt` partiría en un valor cualquiera, con un reset interno de entre 0 y 4095 ciclos, y la RAM partiría con instrucciones arbitrarias. Cuatro construcciones no produjeron ningún mensaje porque son valores o convenciones, no sintaxis:

- `x0` valía 0 solo porque la BRAM se inicializaba en cero y nunca se escribía. En el banco nuevo `x0` no se almacena, la lectura de la dirección 0 entrega 0 por hardware y el write-through de la Sección 4.3 tampoco lo adelanta.
- Los segmentos salían invertidos (`~seg1`) porque el display de la Go Board es activo en bajo. En el macro los puertos quedan activos en alto, que es la función lógica (D8).
- `CYCLES_PER_TENTH = 2500000` en `game.s` supone el oscilador de 25 MHz de la placa. Se corrige para el macro en la Sección 4.4.
- `DebounceDivBits = 15` se eligió para 25 MHz. A 10 MHz da una muestra cada 3,3 ms y acepta un cambio en unos 10 ms, muy por debajo de la décima que muestra el juego, así que se mantuvo (D10).

2 Memoria y programa del juego

El juego usa 115 palabras de instrucciones (460 bytes) y 0 de datos. Las 115 las cuenta el assembler al ensamblar `sw/game.s`: lo informa al terminar y queda como `GAME_WORDS` en la imagen generada. Las 0 de datos salen del programa y de la simulación: todos los `lw` y `sw` usan como base `x2`, que apunta a los periféricos en `0x8000_0000`, y la simulación de la partida completa (`tb/game_tb.v`) no hace ninguna lectura ni escritura en la memoria. Las dos cantidades quedan bajo el máximo recomendado de 256 y 64. Se dejó una RAM de datos de 4 palabras en ambas estrategias (D9): conserva la RAM que nombra la estrategia 1 por 128 flip-flops, y con el mismo valor en las dos la comparación mide solo cómo se guarda el programa.

Extensión del assembler. Se agregaron dos salidas Verilog. `-rom` escribe el módulo `rtl/gen/pochoco_rom.v` completo: un `case` con una palabra por dirección, comentada con su desensamblado, sin reloj ni flip-flops, que entrega 0 fuera del programa. `-vh` escribe `rtl/gen/game_image.vh` con `GAME_WORDS` y la función `game_word(i)`. `make image` regenera ambos desde `game.s`, y siete pruebas nuevas (`assembler/tests/test_image.py`, 80 en total) comprueban que coinciden con el `.hex` y con el `game.s` actual.

Estrategias. En la estrategia 1 (`rtl/pochoco_mem_rom.v`) la ROM entrega la instrucción por dos salidas: una registrada, que conserva la lectura registrada de la RAM del Proyecto 2, y una combinacional (`a_next_o`), con la que el core lee el banco de registros y decodifica un ciclo antes (Sección 4.3). El puerto de datos ve la RAM de 4 palabras en flip-flops. En la estrategia 2 (`rtl/pochoco_mem_ff.v`) instrucciones y datos comparten una memoria de $115 + 4 = 119$ palabras en flip-flops, y el programa entra como valor de reset: cada bit queda como un flip-flop con reset o con set según la imagen, y cualquier palabra se puede escribir desde el puerto de datos, como en el Proyecto 2. Con esta memoria el core usa la temporización anterior, con un ciclo de espera por instrucción (`EarlyRead = 0`). La Tabla 3 compara las dos con el RTL y la configuración del flujo de la corrida final.

	Estrategia 1 (ROM)	**Estrategia 2 (flip-flops)**
Área de celdas de síntesis	156 098 µm²	696 640 µm²
Área de celdas tras place and route	208 439 µm²	977 015 µm²
Flip-flops	964	4 673
Utilización real (pedida: 40 %)	53,5 %	56,2 %
Área del die	0,419 mm²	1,802 mm²
Período mínimo que cierra timing	21,5 ns	$\leq$ 32 ns
Frecuencia máxima	46,51 MHz	$\geq$ 31,25 MHz
CPI del juego completo	1,665	2,665
MIPS (f/CPI)	27,9	$\geq$ 11,7
Nets con violación de antena	0	0

Tabla 3: Estrategias de memoria a $T = 100$ ns (corridas `final` y `s4_t100`) y período mínimo de sus barridos (`f2_*`, con pasos de 0,5 ns cerca del límite, y `s4_*`). En la estrategia 2, 32 ns es el menor período medido: la corrida de 30 ns no termina dentro de los 40 minutos que se fijaron por corrida. Fuente: `docs/datos_informe.md`, sección 3.2.

La estrategia 2 tiene 3 709 flip-flops más, casi todos de los $119 \times 32 = 3\,808$ bits de su memoria unificada. La ROM guarda los mismos 115×32 bits como lógica combinacional y ocupa 7 448 µm², el 4,5 % del diseño (Sección 3). La ruta crítica de la estrategia 2 (`s4_t32`) parte del mismo registro de decodificación y cruza la misma ALU que la de la estrategia 1 (Sección 4.2), pero desde la dirección calculada (`lsu_addr[4]`) sigue por 31 celdas y 17,4 ns de lógica aplanada hasta `_33833_`, un flip-flop con reset; en la estrategia 1 ese último tramo toma 4,4 ns. La ROM combinacional es además la que aprovecha el pipeline de la corrida final. La memoria en flip-flops también tiene esa salida, pero sale del multiplexor de 119 palabras, que quedaría en el camino de la lectura anticipada; como la estrategia 2 no es la de la corrida final, se dejó con el ciclo de espera, lo que además mantiene probado el camino anterior del core. Se eligió la estrategia 1 por área y frecuencia (D17), y con el pipeline rinde además 2,4 veces los MIPS medidos de la 2 en un die 4,3 veces menor.

3 Síntesis

La corrida final aplana el diseño salvo el register file, la ALU y el contador de ciclos, cuya jerarquía conserva con `SYNTH_KEEP_HIERARCHY_MODULES` (D12). El contador se separó en su propio módulo, `pochoco_cycle_counter`, con el mismo comportamiento. Para repartir el área entre todos los módulos se sintetizó además la misma configuración sin aplanar (`SYNTH_HIERARCHY_MODE=keep`, corrida `h3_t100`, solo hasta Yosys), cuyo reporte separa cada módulo. Las dos síntesis difieren solo en que la aplanada optimiza a través de los bordes de módulo, y por eso su área es 5,2 % menor (156 098

contra 164 605 µm^2).

Celda	Cant.	%	Qué parte del diseño la explica
`nand2_1`	934	16,7	NAND de dos entradas, una de las celdas más chicas de la biblioteca (11 µm^2). Yosys, con la estrategia de área, mapea sobre ella la lógica aleatoria: 265 en el register file, 106 en la ALU, 85 en la LSU, 79 en la etapa ID y 74 en la ROM.
`mux2_2`	889	15,9	Multiplexor 2:1. La biblioteca no tiene flip-flops con habilitación, así que cada bit que se escribe bajo condición lleva un mux de retención delante. El banco de registros tiene 796: los 480 de retención de sus bits y el resto en los dos puertos de lectura, donde entran el write-through y la habilitación de lectura. Otros 160 son de la memoria y 117 de la etapa ID (retención del decode registrado y muxes de operandos).
`dffq_1`	779	14,0	Flip-flop sin reset. 544 son del banco de registros ($15\times32=480$ bits más los dos puertos de lectura registrados, 64), 192 de la memoria (registro de instrucción, RAM de 4 palabras y salida de datos), 82 del decode registrado y 2 del offset de byte de la LSU. La síntesis aplanada elimina 41, entre ellos 28 bits del registro de instrucción que el core ya no lee.
`aoi21_1`	437	7,8	Lógica aleatoria: 190 en el register file, 53 en la ALU, 48 en la ROM y 40 en la etapa ID.
`oai21_1`	348	6,2	Lógica aleatoria: 64 en el register file, 49 en la ROM, 48 en la etapa ID y 47 en la ALU.

Tabla 4: Las cinco celdas más usadas de la corrida final (5 577 celdas de 36 tipos). El reparto por módulo sale de la síntesis jerárquica `h3_t100`, que tiene más celdas porque no optimiza a través de los bordes de módulo. Fuente: `asic/sintesis.md`.

Módulo	Instancia	Área (µm^2)	%
`espino_register_file`	`u_soc.u_core.u_rf`	80 349	48,8
`espino_id_stage` (con ALU y decodificador)	`u_soc.u_core.u_id`	29 589	18,0
`espino_alu`	`u_soc.u_core.u_id.u_alu`	10 153	6,2
`pochoco_mem_rom` (con la ROM)	`u_soc.g_mem_rom.u_mem`	27 458	16,7
`pochoco_rom`	`u_soc.g_mem_rom.u_mem.u_rom`	7 448	4,5
`pochoco_periph` (con debounce y contador)	`u_soc.u_periph`	14 409	8,8
`pochoco_cycle_counter`	`u_soc.u_periph.u_cycles`	3 943	2,4
`espino_if_stage`	`u_soc.u_core.u_if`	8 511	5,2
`espino_load_store_unit`	`u_soc.u_core.u_lsu`	2 819	1,7
Bus, sincronizador de reset y control		1 470	0,9
Total		**164 605**	**100**

Tabla 5: Área de celdas por módulo en la síntesis jerárquica `h3_t100`. Las filas con sangría están incluidas en la de arriba. Fuente: `asic/sintesis.md`.

El módulo con más área es el register file (Tabla 5), con 80 349 µm^2: 48,8 % de la síntesis jerárquica y 51,5 % de la final, donde conserva su jerarquía y mide lo mismo. El contador de ciclos ocupa 3 943 µm^2, el 2,4 % de la síntesis jerárquica y el 2,5 % de la final; el 61 % de su área son sus 32 flip-flops con reset y el resto el incrementador.

Antes de sintetizar se esperaba que el banco de registros dominara el área (480 bits sin celdas de memoria en la biblioteca y dos árboles de lectura de 15 a 1) y que los flip-flops fueran la celda más numerosa. Lo primero se cumple; lo segundo no: `dffq_1` queda tercera, detrás de `nand2_1` y `mux2_2` (Tabla 4). El conteo de flip-flops sí coincide con el RTL: 820 sin reset y 185 con reset en la síntesis jerárquica, 964 en la final. La diferencia está en la lógica del banco: con el write-through y

la habilitación de lectura del pipeline (Sección 4.3) pasa de 88 a 265 `nand2_1` respecto del banco sin pipeline, y de 622 a 796 `mux2_2`, en parte por esa lógica y en parte porque la síntesis rehízo el árbol de lectura con menos `mux4_1` (de 128 a 66). Tampoco se esperaba el peso de los multiplexores: casi un `mux2_2` por flip-flop escrito bajo condición, porque la biblioteca no tiene flip-flops con habilitación.

4 Timing

4.1 Barrido de período

La Tabla 6 barre `CLOCK_PERIOD` con el RTL y la configuración de la corrida final, con pasos de 0,5 ns cerca del límite. El diseño cierra hasta 21,5 ns y no a 21 ns, donde falla por 0,129 ns en la esquina lenta. La frecuencia máxima es 46,51 MHz. El rendimiento se mide con la Ley de Hierro: tiempo = instrucciones × CPI ×T, o sea MIPS = f/CPI; con el CPI de 1,665 del juego completo, la corrida final rinde 27,9 MIPS.

	Setup (ns)				**Hold (ns)**					
	`nom_tt`		`max_ss`		`nom_tt`		`max_ss`		**Área**	**Cierra**
T (ns)	WNS	TNS	WNS	TNS	WNS	TNS	WNS	TNS	**(µm²)**	**(9 esq.)**
100	74,404	0	65,725	0	0,442	0	1,065	0	208 439	sí
30	16,460	0	4,924	0	0,442	0	1,065	0	219 059	sí
25	12,306	0	1,579	0	0,442	0	1,065	0	219 531	sí
24	11,931	0	1,755	0	0,442	0	1,065	0	220 655	sí
23	11,272	0	1,494	0	0,442	0	1,065	0	222 714	sí
22	10,358	0	0,656	0	0,442	0	1,065	0	224 767	sí
21,5	10,257	0	0,876	0	0,442	0	1,065	0	231 229	sí
21	9,614	0	−0,129	−0,291	0,442	0	1,065	0	231 638	no

Tabla 6: Barrido de `CLOCK_PERIOD` (corridas `final` y `f2_*`) en la esquina típica `nom_tt_025C_5v00` y la más lenta `max_ss_125C_4v50`. El área es la de celdas estándar tras place and route. Fuente: `metrics.json` y `cierre.md` de cada corrida en `asic/rom/resultados/`.

Cada `CLOCK_PERIOD` puede provocar una nueva optimización del circuito: el resizer cambia el tamaño de las celdas y agrega buffers según el período, y eso cambia el área, el buffering y el placement. Aquí el área de celdas sube en cada punto, de 208 439 µm² a 100 ns a 231 229 µm² a 21,5 ns (11 % más), y por eso el WNS no baja un nanosegundo por cada nanosegundo de período: es 1,755 ns a 24 ns, más que a 25 ns, y 0,876 ns a 21,5 ns, más que a 22 ns. El barrido termina porque el diseño deja de cerrar, no porque el resizer deje de encontrar qué mejorar. El hold no depende del período y es el mismo en los ocho puntos.

El límite también depende de la configuración del flujo. Con la del PDK, el resizer usaba celdas de retardo (`dly*`) como buffers, 12 de ellas en la ruta crítica a 60 ns, y se excluyeron (D15); se excluyó además `buf_1` y se le pidió un margen de setup de 10 ns (D20). El cambio mayor vino de medir cuánto se equivoca el resizer con los cables. Durante el placement y el CTS estima los parásitos con el promedio de las capas del tech LEF, 0,2752 Ω/µm y 64,3 aF/µm en las nueve esquinas. Una regresión lineal de la resistencia y la capacidad que extrae OpenRCX contra el largo ruteado, sobre los 2 399 nets de más de 20 µm de una corrida, da entre 0,498 y 0,599 Ω/µm y entre 148,0 y 188,4 aF/µm según la extracción (min a max): el resizer subestimaba la capacidad de los cables entre 2,3 y 2,9 veces y su resistencia entre 1,8 y 2,2 veces (`asic/modelo_cable.py`, `docs/modelo_cable.md`).

Con parásitos de menos daba un net por cumplido sin poner el buffer, y la violación aparecía recién en el STA final; por eso la corrida final anterior tenía 546 pines con violación de max slew. La corrida final reemplaza esa tabla por los valores del tech LEF multiplicados, capa por capa, por el factor medido en cada extracción (`LAYERS_RC`), y repite la reparación de slew, capacitancia y timing después del ruteo global (`RUN_POST_GRT_DESIGN_REPAIR` y `RUN_POST_GRT_RESIZER_TIMING`). Así quedan 0 violaciones de max slew en las nueve esquinas, a 100 y a 21,5 ns, y una de max capacitancia en la

raíz del árbol de reloj (0,43 pF contra el límite de 0,2 pF del flujo; la biblioteca admite 3,93 pF en ese pin), un pin que la reparación no modifica.

4.2 Ruta crítica

La ruta crítica a 21,5 ns en la esquina lenta (Tabla 7) empieza en el flip-flop `_4136_`, que guarda `alu_op_a_sel[2]` en el decode registrado de la etapa ID, y termina en `_4440_`, el bit 4 de `digit_q`, el registro del display en `pochoco_periph`. Entre medio selecciona el operando A, cruza la ALU hasta `lsu_addr[31]` y sigue por la decodificación de la dirección del bus, combinada con `lsu_we`, hasta la habilitación del registro del display. Es el camino de un store al display: ejecución (la ALU calcula la dirección) y acceso a memoria (el bus decodifica la dirección y habilita la escritura del periférico).

Tramo	Celdas	Retardo (ns)	Llegada (ns)
Árbol de reloj hasta el flip-flop de inicio	3	2,67	2,67
Reloj a salida de `_4136_` (`dffq_4`, etapa ID aplanada)	1	1,83	4,50
Selección del operando A (lógica aplanada de la etapa ID)	9	4,27	8,77
ALU (`u_soc.u_core.u_id.u_alu`) hasta `lsu_addr[31]`	18	9,05	17,82
Decodificación de la dirección y habilitación del registro del display (lógica aplanada)	10	4,42	22,24

Tabla 7: Ruta crítica de `f2_t21p5` en `max_ss_125C_4v50`. El dato debe llegar antes de 23,11 ns (21,5 ns más la latencia del reloj de captura y la corrección por reconvergencia, menos la incertidumbre y el setup): slack de 0,88 ns. Fuente: `asic/rom/resultados/f2_t21p5/max_ss_125C_4v50_ruta_critica.rpt` y `asic/rutas.md`.

La ruta crítica cambia con el período, porque cada período es otro circuito. A 100 ns sale de un bit del PC (`a_addr_i[2]`), cruza la ROM combinacional hasta la instrucción siguiente (`a_next_o[17]`) y el árbol de lectura del banco hasta la salida registrada del puerto A, con 36,03 ns de llegada (`asic/rutas.md`). Es la ruta que agrega el pipeline de la Sección 4.3, que lee el banco en el mismo ciclo del fetch, y a 21,5 ns el resizer la acorta hasta que deja de ser la peor.

4.3 Cambios al RTL

La ruta crítica de partida (`s1_t55`) iba del registro de instrucción al banco de registros pasando por el decodificador, que era su tramo más largo: 19,1 de 55,1 ns. En ese Espino cada instrucción pasaba un ciclo esperando la lectura sincrónica del banco con la instrucción ya estable. El primer cambio (D18) usó ese ciclo para registrar las salidas del decodificador (82 bits: operación y operandos de la ALU, inmediatos, señales de salto, de la LSU y de write-back) en `rtl/espino_core/espino_id_stage.v`, y el decode salió de la ruta de ejecución. Con la configuración del flujo de D15, el WNS a 100 ns subió de 46,76 a 59,75 ns y el período mínimo bajó de 55 a 42 ns, con 4,4 % más de área (Tabla 8).

El segundo cambio (D23) elimina ese ciclo de espera aprovechando que la ROM de la estrategia 1 es combinacional: su salida `a_next_o` tiene la instrucción que pide la etapa IF un ciclo antes de que se cargue en el registro de instrucción. Con `EarlyRead = 1` el core toma de ahí rs1 y rs2 para leer el banco y registra el decode en el mismo flanco en que carga la instrucción, de modo que al llegar a ejecución los operandos y el decode ya están listos. Una instrucción pasa a tomar un ciclo, y dos las cargas y los saltos tomados, contra dos y tres antes (`docs/pipeline.md`). Como la instrucción i escribe su resultado en el mismo flanco en que se leen los operandos de $i+1$, el banco agrega write-through: si en ese flanco se escribe el mismo registro que se lee, distinto de `x0`, la salida registrada toma el dato que se está escribiendo. Durante la detención de una carga el banco no vuelve a leer (`rd_en_i`), y la instrucción siguiente lee el dato cargado por write-through en el flanco en que la carga termina. El CPI del juego completo baja de 2,665 a 1,665.

Con el pipeline, la ruta crítica al período mínimo (`pr_t22p5`, sin el tercer cambio) pasó a ser una carga: del decode registrado por la ALU hasta `lsu_addr[0]`, la alineación del dato leído según los dos bits bajos de la dirección y el mux de write-back hasta el banco, es decir ejecución, acceso a memoria y write-back. El análisis estático la cuenta como de un ciclo, pero en una carga el dato de la memoria llega un ciclo después de emitirla, con el pipeline detenido y la dirección estable. El tercer cambio (B2) registra esos dos bits en la LSU al emitir la carga (`boff_q[1:0]`) y la alineación usa el registro, lo que saca la ruta del análisis sin cambiar el comportamiento. A 22 ns el slack pasa de −0,597 a 0,656 ns y el período mínimo baja de 22,5 a 21,5 ns. En síntesis la LSU crece de 2 711 a 2 819 µm^2 por los dos flip-flops y su lógica, y el área de celdas tras place and route a 100 ns sube 49 µm^2 (0,02 %). Con B2 la crítica pasa a ser la del store al display de la Sección 4.2.

RTL	Flujo	T mín. (ns)	f máx. (MHz)	CPI	MIPS	Área (µm^2)
Sin pre-decodificación	D15	55,0	18,2	2,665	6,8	173 568
Pre-decodificación (D18)	D15	42,0	23,8	2,665	8,9	181 137
Pre-decodificación: corrida final anterior	D20	24,5	40,8	2,665	15,3	182 265
Pipeline y B2	D20	25,5	39,2	1,665	23,6	186 432
Pre-decodificación	D23	22,0	45,5	2,665	17,1	201 309
Pipeline sin B2	D23	22,5	44,4	1,665	26,7	208 390
Pipeline y B2: corrida final	**D23**	**21,5**	**46,5**	**1,665**	**27,9**	**208 439**
B3: comparador de saltos que lee directo del banco	D20	26,0	38,5	2,665	14,4	183 962
B4: ROM y banco combinacionales, una instrucción por ciclo	D20	60,0	16,7	1,312	12,7	158 024
B5: banco de registros con latches, con el pipeline	D20	29,0	34,5	1,665	20,7	165 211

Tabla 8: Período mínimo, CPI del juego completo y MIPS de cada combinación de RTL y flujo; área de celdas tras place and route a 100 ns. Flujo D15: configuración del PDK sin celdas de retardo; D20: además sin `buf_1`, con margen de setup de 10 ns y terminales por función (D21); D23: además el modelo de cable y la reparación después del ruteo global. Pasos del barrido: 0,5 ns en las filas D20 y D23, salvo B3, B4 y B5 (2, 5 y 3 ns), y 2 ns en las filas D15. Fuente: `asic/evolucion.md` y `asic/exploraciones.md`.

Para separar el aporte del RTL del de la configuración del flujo, la Tabla 8 incluye el pipeline con el flujo anterior y el RTL anterior con el flujo final. De los 12,6 MIPS que la corrida final gana sobre la anterior, 8,2 los pone el RTL, 1,7 el flujo y 2,6 la combinación de los dos, que no se puede atribuir a ninguno por separado (`asic/evolucion.md`). El RTL pesa más porque el CPI baja 37,5 % y el período 12,2 %.

Los otros tres cambios de la tabla no se adoptaron. B3 no mueve el período porque la ruta del salto no era la crítica. B4 elimina el ciclo de espera volviendo combinacionales la ROM y el banco, así que todo queda en un ciclo (PC, ROM, decodificador, lectura del banco, ALU y write-back) y el período sube a 60 ns; el pipeline baja el CPI a 1,665 conservando la lectura sincrónica del banco, y su período casi no cambia. B5 ahorra 11 % de área con latches en el banco, pero con la lectura anticipada el pipeline con latches cierra a 29 ns y no a 26, donde cierra sin ellos. La corrida final adopta el pipeline, B2 y el modelo de cable (D23; en D22 se había decidido no incorporar ninguno de los cambios) porque la ganancia en frecuencia y MIPS compensa el área que se agrega, con un die solo 2,5 % mayor (Sección 5.3).

Con la estrategia 2 (`EarlyRead = 0`) el core conserva la temporización anterior y su CPI es el de antes, 2,665. Con la estrategia 1 retira las instrucciones antes, así que la comparación ciclo a ciclo con el RTL anterior ya no es posible, y `make equiv` compara en modo funcional la secuencia de escrituras al banco, de accesos al bus y de salidas. Con `sw/isa_test.s`, que usa todas las instrucciones del assembler, las secuencias son idénticas a las del RTL de referencia (`116fa20`) y a las de la corrida final anterior, con las dos estrategias de memoria. Dos programas nuevos, `sw/hazard_test.s` y `sw/hazard_review.s` (56 y 48 comprobaciones), ejercitan las dependencias entre instrucciones vecinas, y con ellos se detectan las dos mutaciones del RTL que se probaron: el banco sin write-through y la lectura un flanco antes de que termine la carga. En el juego completo ninguna de las

620 217 escrituras al banco usa un valor desconocido (`docs/verificacion/integracion.md`).

4.4 Constantes del juego para $T = 100\,\text{ns}$

Con $T = 100\,\text{ns}$ el reloj es de $f = 1/T = 10\,\text{MHz}$, y una décima de segundo dura

$$\text{CYCLES_PER_TENTH} = 0{,}1\,\text{s} \times 10\,000\,000\,\text{ciclos/s} = 1\,000\,000\ \text{ciclos}.$$

Cambian dos constantes de `game.s`: `CLK_HZ` de 25 000 000 a 10 000 000 y `CYCLES_PER_TENTH` de 2 500 000 a 1 000 000 (D10). `CLK_HZ` documenta la frecuencia y ninguna instrucción la usa. Las esperas están en décimas (`ESPERA_TENTHS = 30` son tres segundos) y no cambian, y el tiempo se sigue mostrando en décimas. `CYCLES_PER_TENTH` se carga con `li x12` en `cyc2tenths` y en `wait_tenths`, y cada `li` se expande en `lui` más `addi`: 1 000 000 = 0x000F4240, así que `lui x12, 0xF4` y `addi x12, x12, 576` (el bit 11 de `0x240` es cero y no hace falta corregir la parte alta). Al regenerar el programa con `make image` cambian cuatro palabras de la ROM (Tabla 9).

Dirección	25 MHz	10 MHz	Instrucción a 10 MHz	Subrutina
0x0148	00262637	000f4637	lui x12, 0xf4	cyc2tenths
0x014c	5a060613	24060613	addi x12, x12, 576	cyc2tenths
0x019c	00262637	000f4637	lui x12, 0xf4	wait_tenths
0x01a0	5a060613	24060613	addi x12, x12, 576	wait_tenths

Tabla 9: Palabras de la ROM que cambian al pasar de 25 a 10 MHz. Fuente: `rtl/gen/pochoco_rom.v`; las constantes cambiaron en el commit `5d57856`.

5 Floorplan, red de alimentación y placement

5.1 Utilización y área mínima

Se barrió `FP_CORE_UTIL` de 40 a 90 % con $T = 100\,\text{ns}$ (Tabla 10 y Figura 1). Entre las probadas, el área mínima con la que el diseño completa el flujo y cumple todos los criterios es la de 73 %: un die de 476,1 × 494,1 µm, 0,235 mm², 43,9 % menos que el de la corrida final. A 74 y 75 % el placement detallado ya no encuentra lugar legal para todas las celdas durante la reparación después del ruteo global; a 78 % le pasa lo mismo dentro del CTS, a 80 % en la reparación después del placement global y a 90 % las celdas ya no caben en el core. La utilización real queda entre 13,5 y 19,7 puntos sobre la pedida porque el core se dimensiona con el área de síntesis y después se agregan celdas de tap, el árbol de reloj y los buffers y celdas más grandes que pone el resizer.

Con la utilización suben las violaciones de max slew, de 0 a 40 % (8 a 50 % y 0 a 60 %) a 20, 92 y 99 pines a 70, 72 y 73 %, y las de max capacitancia de 1 a 5 y 6: con el core casi lleno la reparación ya no tiene dónde poner buffers. El largo de cables del ruteo detallado baja hasta 60 % (de 377,5 a 352,2 mm) y desde ahí sube hasta 449,4 mm a 73 %, aunque el die es menor, porque el router tiene que desviar cables. Las violaciones de DRC al inicio del ruteo detallado pasan de 2 a 40 % a entre 2 189 y 3 252 desde 50 %, y el router las lleva a cero en todas las corridas que terminan. El die mínimo (73 %) y el mínimo de cables (60 %) no coinciden: por costo conviene el primero, por calidad del ruteo el segundo.

5.2 Terminales del macro

Los terminales se ubicaron por función con `IO_PIN_ORDER_CFG` (`asic/rom/pines.cfg`, D21), como se ve en el borde de la Figura 3. Al oeste van las entradas: `btn[3:0]`, `clk` al centro del lado y `rst_n`. Al este van los dos displays, cada bus contiguo y en orden de bits; al norte los LEDs y al sur el SPI. Cada grupo llegaría así a pads vecinos de un chip completo sin cruzarse, y las señales entran por un

Pedida	Die (µm)	Die (mm²)	Real	WNS (ns)	Resultado
40 %	638,5 × 656,4	0,419	53,5 %	65,73	cierra
50 %	572,5 × 590,4	0,338	65,3 %	63,70	cierra
60 %	523,8 × 541,7	0,284	76,9 %	65,12	cierra
70 %	485,9 × 503,9	0,245	88,7 %	60,80	cierra
72 %	479,3 × 497,3	0,238	91,7 %	61,35	cierra
73 %	476,1 × 494,1	0,235	91,8 %	59,53	cierra
74 %					falla tras el ruteo global (18 instancias)
75 %					falla tras el ruteo global (77 instancias)
78 %					falla en CTS (36 instancias)
80 %					falla tras el placement global (5 instancias)
90 %					falla en placement global (`GPL-0301`)

Tabla 10: Barrido de utilización a 100 ns (corridas `final` y `u2_*`), con el WNS de setup en `max_ss_125C_4v50`. Las cuatro fallas de placement detallado son `[DPL-0036] Detailed placement failed`, con las instancias que quedan sin lugar; a 90 %, `[GPL-0301] Utilization 100.620 % exceeds 100%`. Fuente: `metrics.json` y `falla.txt` de cada corrida en `asic/rom/resultados/`, y `docs/datos_informe.md`, sección 3.5, para el paso en que falla 75 %.

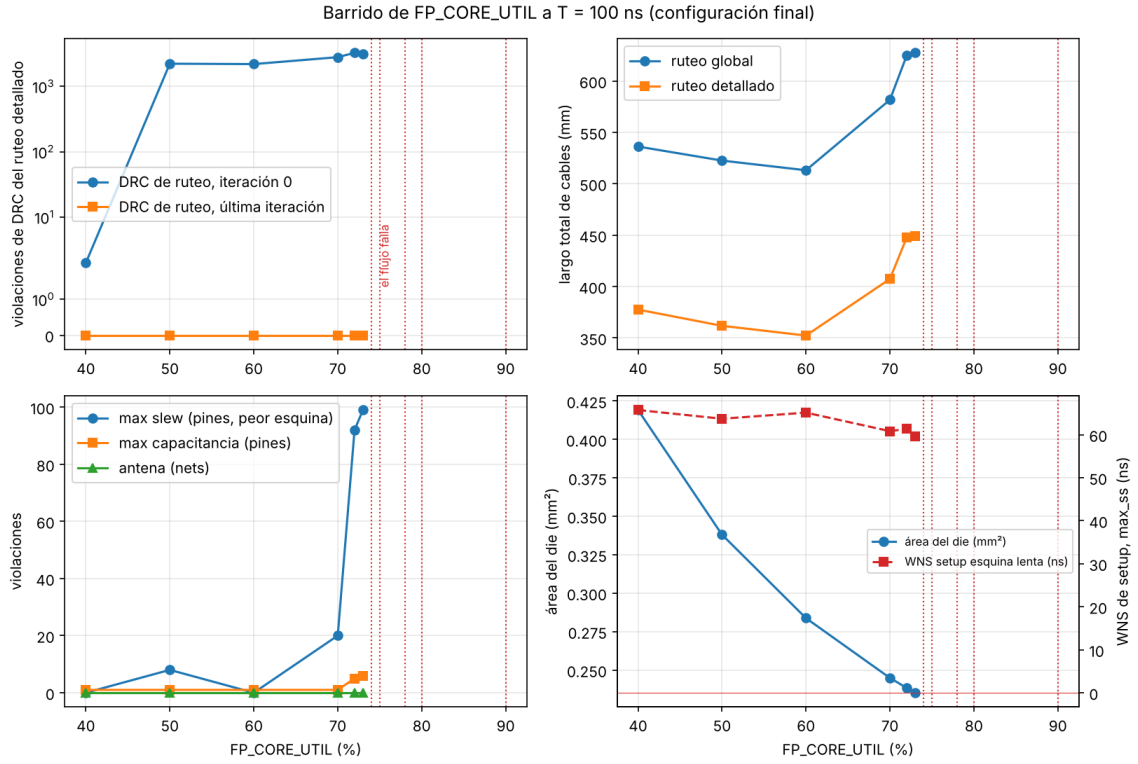


Figura 1: Utilización pedida contra violaciones de DRC del ruteo detallado, largo de cables, violaciones de slew, capacitancia y antena, área del die y WNS. Las líneas punteadas marcan las corridas que no terminan. Fuente: `asic/graficos.py`.

lado y salen por el opuesto. Con la corrida final anterior se dejó que el flujo los ubicara (corrida `u1_auto`): puso `clk` cerca de una esquina, llevó los botones al sur junto a tres de los cuatro terminales de SPI, juntó los LEDs con los displays en el lado este y desordenó los bits dentro de cada bus, y ahorró solo 0,4 % de cable con el mismo WNS.

Estos terminales son rectángulos de Metal3 (oeste y este) y Metal2 (norte y sur) en el borde del macro, y solo conectan lógica. Un chip completo rodearía el macro con un anillo de pads de I/O: celdas grandes con protección contra descargas electrostáticas, buffers de salida capaces de entregar la corriente de un LED o un display, y entradas con histéresis.

5.3 Costo

Con US$23 000 por mm^2 y un costo proporcional al área del die, la corrida final (0,419 mm^2) costaría US$9 640, la de 73 % de utilización (0,235 mm^2) US$5 410 y la estrategia 2 (1,802 mm^2) US$41 437. La aproximación ignora que un prototipo se paga por cupo en una oblea multiproyecto con un área mínima, que el macro no incluye pads, anillo de sello ni encapsulado, que el precio por mm^2 baja con el volumen y que las máscaras son un costo fijo. Tampoco considera el rendimiento de fabricación, que baja con el área, así que el costo real por chip crece más rápido que el área.

La decisión con más impacto en área y costo fue la estrategia de memoria: con el mismo RTL y la misma configuración, la estrategia 2 necesita un die 4,3 veces mayor (Tabla 3). La siguiente es la utilización, que de 40 a 73 % reduce el die en 43,9 %. Los cambios de D23 suben el área de celdas 14,4 % (de 182 265 a 208 439 µm^2: 2,3 % el pipeline con B2 y 11,8 % el modelo de cable), pero el die crece solo 2,5 % (de 0,409 a 0,419 mm^2, US$234 más). `FP_CORE_UTIL` dimensiona el die sobre el área de síntesis, que sube 2,6 % con el pipeline, y los buffers que agrega el resizer caben adentro: el modelo de cable solo sube la utilización real, de 47,9 a 53,5 %.

6 Signoff y layout

6.1 DRC, LVS y antenas

La corrida final no tiene violaciones de DRC (Magic), diferencias de LVS ni nets con violación de antena, y el flujo insertó un diodo de antena (Tabla 1). Además se corrió sobre el GDS final el deck de DRC de KLayout de GlobalFoundries (684 reglas, `asic/drc/`), que LibreLane salta con este PDK, y no marcó nada. Para comprobar que el deck detecta errores, se agregaron a una copia del GDS dos violaciones de Metal1: un trazo de 0,20 µm de ancho y dos trazos separados por 0,15 µm, contra mínimos de 0,23 µm. El deck marcó exactamente esas dos, en las reglas M1.1 y M1.2a y en sus coordenadas (`asic/rom/resultados/final/drc_klayout/control_positivo.md`).

Durante la comparación de estrategias, antes del pipeline, apareció una violación de antena en la estrategia 2, en todas sus corridas. En `s2_t100`, el net `net3342` reparte una señal de alto fanout entre cinco buffers y ocupa una envolvente de 666 × 364 µm (Figura 2). En el pin `fanout3341/I`, la razón entre el área lateral del Metal2 conectado (perímetro por espesor) y el área de la compuerta es 449,1, 1,12 veces el máximo de 400 (`asic/ff/resultados/s2_t100/antenna.rpt`). Durante la fabricación, un tramo largo de metal conectado solo a compuertas acumula carga del grabado por plasma, y puede romper el óxido de la compuerta antes de que las capas superiores lo conecten a una difusión que la descargue.

Se probaron dos soluciones (Tabla 11). La inserción heurística de diodos (`RUN_HEURISTIC_DIODE_INSERTION`) elimina la violación pero inserta 36 385 diodos, con 18 % más área de celdas y 16 ns menos de slack. Darle más iteraciones y margen a la reparación de antenas del ruteo global (`GRT_ANTENNA_REPAIR_ITERS = 6`, `GRT_ANTENNA_REPAIR_MARGIN = 30`) también la elimina, con 108 diodos más y sin aumentar el área. Con el core y la configuración de la corrida final, la estrategia 2 ya no tiene la violación (`s4_t100`: 0 nets en violación), y la estrategia 1 no la tuvo en ninguna corrida.

Corrida	Violaciones	Diodos	Área de celdas (µm^2)	WNS (ns)
`s2_t100` (original)	1	150	838 173	39,65
`s2_t100_diodos`	0	36 385	992 171	23,65
`s2_t100_antrep`	0	258	833 559	30,03

Tabla 11: Violación de antena de la estrategia 2 con el RTL anterior al pipeline y sus dos soluciones: nets con violación, celdas de antena, área de celdas y WNS de setup en `max_ss_125C_4v50`. Fuente: `metrics.json` y `antenna_summary.rpt` de cada corrida en `asic/ff/resultados/`.

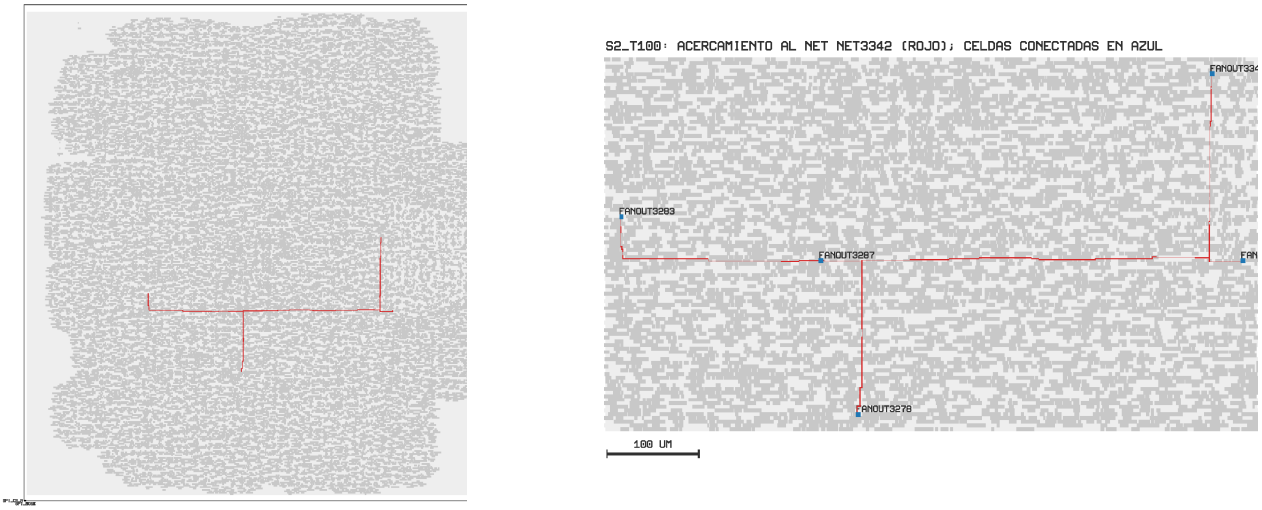


Figura 2: Net `net3342` de `s2_t100` (rojo) sobre el layout de la estrategia 2, completo y en acercamiento, con los buffers que conecta en azul. Fuente: `asic/layout/anotar.py`.

6.2 Layout anotado

El register file, la ALU y el contador de ciclos conservan su jerarquía en la síntesis (Sección 3), así que sus celdas llegan al DEF final con el nombre de su instancia como prefijo: `u_soc.u_core.u_rf/`, `u_soc.u_core.u_id.u_alu/` y `u_soc.u_periph.u_cycles/`. Un script de KLayout (`asic/layout/anotar.py`) lee el DEF con los LEF del PDK, colorea las celdas de cada prefijo y genera la Figura 3 y una base de marcadores (`bloques.lyrdb`) para repetir las marcas en la interfaz de KLayout (`docs/layout.md`).

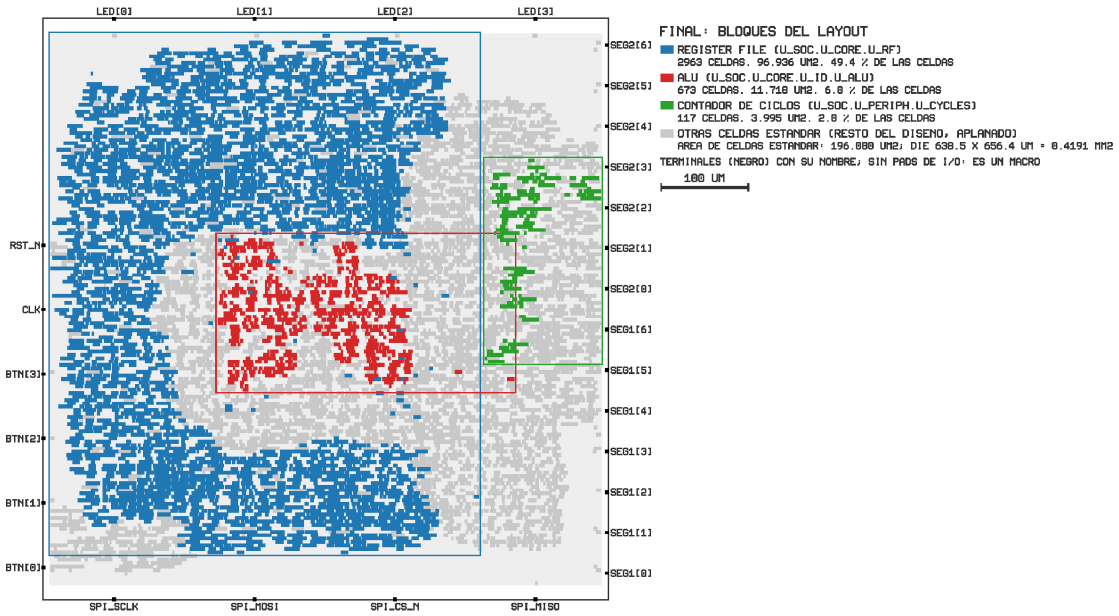


Figura 3: Layout de la corrida final con el register file (azul), la ALU (rojo) y el contador de ciclos (verde), cada uno con su rectángulo envolvente, y los terminales en el borde. Fuente: `asic/rom/resultados/final/layout/`.

El register file se reparte en una herradura que ocupa el oeste del core y sus franjas norte y sur, alrededor de la ALU, que queda al centro; el contador de ciclos queda en una franja cerca del borde este. Las áreas de la figura son mayores que las de síntesis (96 936 contra 80 349 µm^2 en el register file) porque el resizer agrega buffers dentro de cada módulo, y sus porcentajes son sobre 196 080 µm^2

de celdas lógicas, sin las de tap ni las de borde.

7 Simulación funcional post-síntesis

`make post-synth` simula el RTL y el netlist de celdas estándar con el mismo testbench y los mismos estímulos, y compara las trazas. El netlist sale de Yosys con la configuración final (`asic/sim/config.yaml`, que solo cambia la ROM y el debounce), y las celdas se simulan con los modelos Verilog del PDK compilados sin bloques `specify` (`-DFUNCTIONAL`), o sea sin retardos. El testbench (`tb/post_synth_tb.v`) usa solo los puertos de `asic_top`, aprieta los botones en ciclos absolutos leídos de `asic/sim/estimulos.txt` y escribe una línea por cada cambio de los LEDs o del display.

Para que la simulación dure miles de ciclos y no millones, las dos usan el mismo `game.s` ensamblado con `-D CYCLES_PER_TENTH=1`, con lo que el display muestra directamente los ciclos medidos, y el debounce con `DebounceDivBits = 2`, que acepta un botón en unos 12 ciclos (`asic/sim/README.md`). El LED objetivo de cada ronda lo elige el LFSR del programa, así que los estímulos se calibraron corriendo el RTL por etapas y leyendo en su traza qué LED se encendió y cuándo (`asic/sim/calibrar.py`). Cada pulsación llega 20 ciclos después de encenderse el LED y dura 60.

Ciclos	Botón	Qué pasa	Ciclo
300 a 360	`0001`	Pulsación de inicio; empieza la ronda 01	404
635 a 695	`0010`	Ronda 01, LED `0010`: acierto, el display muestra 51	1 033
1 373 a 1 433	`0010`	Ronda 02, LED `0001`: botón incorrecto, el display muestra EE	1 405
		La ronda vuelve a 02, no avanza	1 507
1 733 a 1 793	`0001`	Reintento de la ronda 02: acierto, el display muestra 51	2 131

Tabla 12: Escenario simulado. La última columna es el ciclo en que aparece cada resultado, igual en el RTL y en el netlist. Fuente: `asic/sim/resultado.md`.

Las dos trazas son idénticas, con 17 líneas (la inicial y 16 cambios de salida) en 4 500 ciclos y ningún valor desconocido (X) en las salidas del netlist. La ronda correcta mide 51 ciclos en las dos simulaciones, y la pulsación incorrecta muestra EE y repite la ronda 02 en vez de pasar a la 03 (Tabla 12). Como control negativo, `make post-synth-control` atrasa 8 ciclos la pulsación de la ronda 01 solo en el netlist: el display muestra 59 en vez de 51 y la comparación da FAIL. Con 7 ciclos de atraso la traza no cambia, porque el programa lee los botones en un lazo de 8 ciclos y la medición no distingue pulsaciones dentro de la misma vuelta. Con las constantes reales, el netlist de la corrida final también coincide con el RTL durante 320 000 ciclos, hasta los cuatro LEDs y el 01 de la primera ronda, en los ciclos 262 151 y 262 172 (`make post-synth-final`). Además, el flujo de equivalencia formal de Yosys prueba que el RTL es equivalente a los netlists de síntesis y post-PnR (976 de 976 y 968 de 968 celdas `$equiv`, `docs/equivalencia.md`).

La simulación opcional con retardos usa el netlist y el SDF de una corrida completa con la configuración de simulación (`d2_pnr`, que cierra) y el mismo escenario, en las esquinas típica, lenta y rápida, con tres ejecuciones por esquina en dos simuladores. Icarus Verilog 13.0 anota los retardos pero no evalúa los timing checks; Open Verilog CVC 7.00b sí evalúa los 7 173 checks del SDF (`$setup`, `$hold`, `$width`, `$recovery`, `$removal` y `$period`). Las 18 trazas son idénticas a la del RTL, sin valores X, y CVC no reporta ninguna violación. Como control, aplicar las entradas 1 ns después del flanco produce 8 violaciones en los cuatro flip-flops que reciben una señal externa que cambia en el escenario (los dos del sincronizador de reset y los de los botones 0 y 1), porque el reloj les llega unos 1,4 ns después del flanco externo; por eso el escenario con SDF aplica las entradas a mitad de ciclo. El netlist de la corrida final con su SDF de la esquina típica también reproduce al ciclo los 320 000 ciclos del escenario con las constantes reales (`asic/sim/resultado_sdf.md`).